

Curate — Technology Stack

Frontend

Framework

- React 19
- TypeScript
- Vite

Styling

- Tailwind CSS v4

UI Components

- shadcn/ui

Animation

- Framer Motion

Routing

- React Router DOM

Forms

- React Hook Form
- Zod

State Management

- TanStack Query
- Context API

HTTP Client

- Axios

Icons

- Lucide React
-

Backend

Runtime

- Node.js

Framework

- Express.js

Language

- TypeScript

Architecture

- REST API
-

Database

PostgreSQL

Hosted by Supabase

Authentication

Supabase Authentication

- Email Authentication
- Session Management
- Password Reset
- Email Verification
- Role-Based Authentication

Roles

- Space Owner
 - Artist
 - Internal Staff
-

Storage

Supabase Storage

Used for

- Artwork Uploads
 - Artist Profile Images
 - Space Images
 - Documents
 - Invoices
-

File Upload

Base64 Upload Flow

Client

↓

Base64 Encoding

↓

Async Upload

↓

Supabase Storage

↓

Public URL Generated

↓

Database Updated

Current Decision

- No image optimization pipeline
- No Sharp
- No thumbnail generation
- Keep uploads asynchronous and simple for MVP

Future optimization can be added later without changing the upload flow.

Email

Supabase SMTP

Used for

- Registration
- Email Verification
- Password Reset
- Order Confirmation
- Payment Confirmation
- Installation Updates
- Artist Notifications

Payments

MVP

Dynamic QR Code

↓

Customer Payment

↓

Payment Verification

↓

Payment Success

↓

Order Created

Failure Flow

QR Generated

↓

Payment Timeout

OR

Verification Failed

↓

Payment Failed

↓

Retry Payment

The payment layer should be abstracted behind a **Payment Service**, allowing the QR implementation to be replaced later without affecting the rest of the application.

Future Upgrade

Stripe Mock Payments

↓

Stripe Production

or

Razorpay

The UI and business flow should remain unchanged regardless of the provider.

Notifications

In-App Notifications

Examples

- Artwork Selected
- Upload Approved
- Payment Received
- Installation Scheduled
- Rotation Reminder
- Order Completed

No email-only notification dependency.

Every important event should also appear inside the product.

Analytics

Keep analytics lightweight and action-oriented.

Dynamic Dashboard Analytics

Space Owners

- Active Installations
- Current Collection
- Upcoming Rotation
- Orders
- Payment History

Artists

- Selected Works
- Active Installations
- Pending Reviews
- Earnings
- Notifications

Internal

- Revenue
- Orders
- Installations
- Artist Growth

No unnecessary charts or enterprise BI dashboards.

Only surface metrics that help users make decisions.

Validation

Zod

API Communication

Axios

REST APIs

Motion

Framer Motion

Purpose

- Page Transitions
- Micro Interactions
- Loading States
- Navigation Feedback

Motion communicates state.

It should never become decoration.

Architecture Principles

Feature-Based Folder Structure

Reusable Components

Shared Hooks

Shared Services

Design Tokens

Role-Based Routing

Protected Routes

Responsive by Default

Accessibility First

Design Philosophy

The interface should feel like entering a thoughtfully curated space—not operating business software.

Design should prioritize:

- **Photography over decoration**
- **Typography over visual noise**
- **Whitespace over crowded layouts**
- **Clear decisions over excessive options**
- **Calm interactions over flashy animations**

The UI should quietly support the experience rather than compete with the artwork.